

CSPCL: A Stepwise, Non-Disruptive Transition to Delay-Tolerant Networking for CubeSat Operators

Mathias Durat[†], Sarah Theoullé[†], Hugo Ponthieu[†], Tristan-Mihai Radulescu[†], Olivier De Jonckère^{*†},

^{*} LIRMM, Université de Montpellier, France

[†] Polytech, Université de Montpellier, France

Abstract—As Delay-Tolerant Networking (DTN) gains increasing traction in the space community, CubeSat operators may face the challenge of adopting DTN capabilities without disrupting existing software stacks built around the CubeSat Space Protocol (CSP). Rather than proposing a full architectural shift, our goal is to offer operators a gentle introduction to DTN, allowing them to experiment with its capabilities while preserving their existing infrastructure. This paper reports on our experience assembling and validating such a non-intrusive full stack in a representative CubeSat context. We evaluate interoperability with three BPA implementations: Hardy, μ D3TN, and UNIBO-BP, with A-SABR contact-based routing integrated into two of them. Rather than replacing the existing CSP infrastructure, CSPCL operates as an underlayer, transmitting DTN bundles over CSP to enable DTN capabilities on top of an unmodified CSP network. For the existing applications requiring CSP-to-CSP communication across DTN boundaries, we introduce Charon, a network proxy that encapsulates IP and CAN traffic over the Bundle Protocol, allowing legacy applications to benefit from DTN features transparently. The full stack is validated on two PolarFire RISC-V boards communicating over a CAN bus, mirroring the hardware and transport layer of the Centre Spatial de l'Université de Montpellier's upcoming 12U CubeSat mission. Our results demonstrate end-to-end feasibility on representative flight hardware, and offer a concrete and accessible integration path for CubeSat operators considering a progressive adoption of DTN.

Index Terms—Delay-Tolerant Networking, CubeSat, Convergence Layer Adapter,

I. INTRODUCTION

Delay-Tolerant Networking (DTN) has emerged as a key technology for future space communication systems [1]. Deep-space networks operate under challenging conditions, including long propagation delays, intermittent connectivity, and constrained data rates resulting from extreme distances and varying link availability. Consequently, the Interagency Operations Advisory Group (IOAG) recommends DTN capabilities for most future surface and orbital assets [2]. This vision is reflected in major space communication initiatives such as NASA's Lunar Communications Relay and Navigation Systems (LCRNS) [3], ESA's Moonlight program [4], [5], and the NASA-ESA LunaNet interoperability architecture [6], [7], [8], [9].

Contacts: Communication opportunities in space are largely predictable but inherently intermittent due to orbital dynamics, planetary rotation, and line-of-sight constraints. DTN

models these opportunities using the concept of a *contact*, defined as a scheduled unidirectional transmission opportunity characterized by a start time, an end time, and a nominal data rate. Together, these parameters determine both the available transmission *volume* and the expected propagation *latency* of the contact.

Bundle Protocol: At the core of DTN lies the Bundle Protocol (BP) [10], [11], which provides end-to-end data delivery through a store-and-forward architecture. Data are encapsulated into *bundles*, which may be stored for extended periods before transmission opportunities become available. Unlike conventional Internet protocols that assume relatively continuous connectivity, BP supports asynchronous hop-by-hop forwarding, allowing communication across highly disrupted networks without requiring immediate downstream availability.

Bundle Protocol Agent: A Bundle Protocol Agent (BPA) is a software entity that implements the Bundle Protocol (BP) and is responsible for managing the full lifecycle of bundles. This includes generating bundles from application data, storing them when no forwarding path is immediately available, and forwarding them when a contact arises. The BPA handles endpoint ID (EID) addressing, interacts with a routing module (such as SABR) to determine the next hop for each bundle, interfaces with underlying layers to transmit and receive bundles over underlying physical links, and discards bundles that exceed their time-to-live.

Convergence Layer Adapters: To facilitate deployment across heterogeneous communication infrastructures, the Bundle Protocol relies on *Convergence Layer Adapters* (CLAs), which map bundle transmission onto underlying transport protocols. A CLA enables a Bundle Protocol Agent (BPA) to delegate the actual transmission and reception of bundles to a lower-layer protocol while preserving BP semantics.

For example, a BPA deployed over the terrestrial Internet may use a TCP-based CLA, allowing BP endpoints to communicate through conventional TCP/IP networks. Although BP provides its own routing and quality-of-service mechanisms, such deployments can leverage existing Internet routing and transport services transparently, while the communicating BPAs perceive each other as directly connected DTN neighbors.

An Overlay Architecture: The combination of the Bundle Protocol and CLAs enables DTN to operate as an overlay architecture spanning heterogeneous network environments.

This work was supported, in whole or in part, by the French National Research Agency (ANR) through project ANR-25-CE25-4175-01.

Different segments of an end-to-end communication path may employ distinct protocol stacks while remaining interconnected at the BP layer.

For instance, terrestrial segments may transport bundles over TCP/IP, whereas space links may rely on the Licklider Transmission Protocol (LTP). A gateway node equipped with both TCP- and LTP-based CLAs can seamlessly transfer bundles between these environments, allowing a unified DTN architecture to bridge conventional terrestrial networks and space communication systems.

Ring-Road Networks and Small Satellite Missions: Although DTN was initially developed to address the challenges of deep-space communications, its principles are equally relevant to smaller-scale networks characterized by predictable disruptions and intermittent connectivity. DTN has already demonstrated operational value aboard the International Space Station (ISS), where it is used for the reliable retrieval of scientific payload data.

For smaller operators, the *ring-road* concept [12] offers a cost-effective approach to providing connectivity over large geographical areas. In this architecture, a limited number of DTN-enabled satellites relay data between isolated ground locations—such as islands, remote research stations, or oceanic buoys—and Internet-connected gateways. This approach was successfully demonstrated during the OPS-SAT mission [13] and can be extended to a wide range of Earth-observation and Internet-of-Things applications.

II. BACKGROUND AND MOTIVATION

The Centre Spatial de l'Université de Montpellier (CSUM) develops and operates CubeSats built on the CubeSat Space Protocol (CSP). Integrating Delay/Disruption Tolerant Networking (DTN) capabilities into these systems would enable interoperability with emerging space networking infrastructures while preserving compatibility with existing mission architectures. Since a non-intrusive deployment of DTN must overlay the Bundle Protocol (BP) over CSP, a dedicated Convergence Layer Adapter (CLA) is required, bridging BP-based DTN networks with CSP-based CubeSat systems and enabling delay/disruption-tolerant communications. We therefore introduce the CubeSat Space Protocol Convergence Layer (CSPCL) adapter. The Adaptive Library for Schedule-Aware Bundle Routing (A-SABR) [14] targets both research and operational activities and is designed to instantiate algorithm variants derived from the Schedule-Aware Bundle Routing standard of the Consultative Committee for Space Data Systems [15]. While simpler routing approaches could be considered, a software-based implementation offers significant flexibility in design and provides a suitable framework for increasing the Technology Readiness Level (TRL) of the A-SABR library. Finally, delay/disruption tolerance is provided only at the Bundle Protocol (BP) layer above CSP and is therefore not directly accessible to existing applications operating at the CSP layer.

To enhance the effectiveness of the experimental setup, we are working on Charon, a proxy mechanism that will

encapsulate CSP packets over BP for end-to-end transport and subsequently decapsulates them, restoring the original CSP communication flow beneath the BP overlay at the destination. Although this approach may appear inelegant, it offers two key advantages: it preserves the requirement of leaving the underlying CSP network stack unchanged at CSUM, and it enables operators to experience DTN capabilities using existing applications. Works like the BP socket [16] or Charon contributes to lower the transitional efforts toward DTN for wider adoption.

CSUM's CubeSat avionics rely on the Controller Area Network (CAN) bus as the primary inter-component communication device. CSP operates directly over CAN, making it the natural intra-satellite link layer that any non-disruptive DTN integration must accommodate.

III. CUBESAT SPACE PROTOCOL CONVERGENCE LAYER

A. Convergence Layer Adapter Library for CSPCL

CSPCL stands for CubeSat Space Protocol Convergence Layer. It is the implementation of a CLA using CSP as communication protocol. The CSPCL aims to regroup and aggregate all the methods needed for a BPA to use CSP as a Convergence Layer. Its goal is to have a common base (unlike the current implementations of the different CLAs that are targeted to an implementation of BP and developed independently) that will allow an easier integration in any BPA. While other CLAs such as TCPCL are backed by an RFC to specify them so that any BP agent implementation exhibits the same behaviour, our CSPCL proposal consists of a common library in order to unify behaviour. This approach aims to demonstrate interoperability between different BPAs while operating CSPCL, with a formal RFC potentially following in the future to standardize the protocol.

The implementation of the CLA takes advantage of the features implemented in CSP, which are RDP (Reliable Datagram Protocol) enabling reliable CSP connections while using ACKs. It also uses SFP (Small Fragmentation Protocol), which allows fragmentation of the payloads transmitted (the size of a Bundle Protocol packet can greatly exceed the MTU of a CSP packet). For RDP to work, we need a connection between the two BPAs, in that way it is the same as what is done in the TCPCL: the CSPCL opens a connection used for the whole contact, and we assume the link is broken between two nodes if the connection breaks. The connections are managed with a pool allowing connections to live during the entire contact. The outbound pool uses Least Recently Used (LRU) eviction policy with a monotonic tick counter to track access patterns. When the pool is full, the least-accessed connection is closed to make room for new peers, while frequently-contacted destinations benefit from persistent cached connections. Thread-safe atomic operations via mutex locks ensure concurrent senders safely share the pool without duplication. An optional age-based invalidation mechanism (configurable via environment variable) allows stale connections to be pruned on-the-fly, enabling graceful adaptation to network topology changes while minimizing setup overhead

for repeated communications. The main feature of CSP is to work on top of a Controller Area Network (CAN) bus. CAN is a robust, low-bandwidth serial bus protocol widely adopted in CubeSat avionics as the primary inter-component communication medium, enabling reliable data exchange between attitude determination and control systems (ADCS), power management units (PMU), and scientific payloads. CSPCL integrates CAN as a physical transport layer via the SocketCAN interface, allowing CSP packets to be framed and transmitted over CAN, providing automatic error detection via CRC checksums, collision avoidance through priority-based arbitration, and guaranteed message delivery—critical properties in radiation-prone space environments where transient faults are common. CAN’s low power consumption and minimal wiring overhead (two wires vs. dedicated point-to-point links) make it ideal for CubeSats operating under severe mass, volume, and electrical power constraints. By layering CSP/Bundle Protocol over CAN, CSPCL bridges satellite communications standards with the proven, space-flight-qualified CAN hardware ecosystem, enabling CubeSats to employ standardized DTN bundles for store-and-forward messaging while leveraging battle-tested onboard bus infrastructure.

B. CLA for unibo-bp

Unibo-BP is a C++ implementation of Bundle Protocol version 7 (BP7) [10] developed at the University of Bologna as part of the broader Unibo-DTN project [17]. Its architecture is built around modularity: convergence layers are not compiled into the BPA but run as independent processes, communicating with the core bundle processor over a UNIX domain socket through the `CLAOVERIPC` mechanism.

This process-isolated design fundamentally shapes how CSPCL integrates with Unibo-BP. For Unibo-BP, CSPCL is packaged as a standalone C daemon, that links both `libcspcl` and `libunibo-bp-api`, the C wrapper Unibo-BP exposes over its internal C++ interface. At startup, the daemon calls `unibo_bp_cla_connect()` with the path to the BPA’s runtime directory, where the library opens the socket and performs the initial CLA handshake (setting a CLA identifier and name). A running BPA paired with a separately spawned CLA daemon highlights a key property of `libcspcl`: the library is BPA-agnostic. Multiple integrations of `libcspcl` exist, each coupling the same C library to its host BPA through whatever mechanism that BPA’s architecture prescribes; for Unibo-BP, that mechanism is an IPC-connected process rather than an in-process link.

Control flows in both directions over this IPC channel. The BPA calls into the daemon through registered C callbacks: `on_outbound_pdu` triggers outbound transmission, `on_add_contact` and `on_remove_contact` open and close links in response to the contact schedule, and `on_stop` signals a clean shutdown. The daemon pushes data back by calling `unibo_bp_cla_inbound_pdu`, `unibo_bp_cla_transmission_success`, and `unibo_bp_cla_transmission_failure`. A notable design choice specific to Unibo-BP is that bundles are

exchanged as file descriptors rather than in-memory buffers. On the outbound path, `on_outbound_pdu` receives an open file descriptor from which it reads the serialised bundle before handing it to `cspcl_send_bundle()`. On the inbound path, the daemon calls `unibo_bp_cla_open_file()` to obtain a writable file descriptor, writes the bundle bytes received from the CSP receive loop, then submits the descriptor via `unibo_bp_cla_inbound_pdu()`.

Link management is dynamic and event-driven. The daemon maintains a peer table keyed by 8-bit CSP node address. A link is created on demand either when an inbound packet arrives from a previously unknown CSP address, or when `on_add_contact` fires because an entry has been added to the contact plan. Each peer is assigned a BP endpoint using the IPN scheme, with the CSP node address used directly as the IPN node number (e.g., CSP address 2 maps to `ipn:2.0`), consistent with the address mapping convention described for the other BPA integrations. When a contact ends, `on_remove_contact` closes the corresponding link.

C. CLA for μ D3TN

μ D3TN (micro Delay/Disruption Tolerant Networking) is a lightweight, production-grade C implementation of Bundle Protocol version 7 (BP7) designed for resource-constrained embedded systems and space applications. Unlike Unibo-BP’s process-isolated architecture or Hardy’s FFI-based approach, μ D3TN integrates convergence layers directly as in-process modules through an event-driven callback mechanism, making it particularly suitable for embedded and microcontroller deployments. CSPCL integrates into μ D3TN as a native convergence layer, registered at startup to handle bundle transmission and reception through the BPA’s internal event loop. Contact management is event-driven: when contacts are scheduled or closed, μ D3TN notifies CSPCL to establish or tear down corresponding CSP connections. On the outbound path, the BPA invokes CSPCL to send serialised bundles directly without intermediate buffering; on the inbound path, CSPCL receives bundles through CSP and delivers them to the BPA. Because the integration operates entirely within the BPA’s execution context, there is no inter-process communication overhead or separate daemon process—bundles are exchanged as in-memory buffers, making the integration lean and power-efficient for constrained platforms. In the experimental setup, μ D3TN instances serve as gateway nodes, bridging external IP traffic through Charon into the DTN overlay while leveraging A-SABR for routing decisions.

D. CLA for hardy

Hardy is an asynchronous Rust implementation of Bundle Protocol version 7 (BP7) [10], structured as a Cargo workspace whose core Bundle Protocol Agent (BPA) logic resides in a dedicated crate. As with the other BPA implementations considered here, CSPCL is integrated into Hardy as a *Convergence Layer Adapter* (CLA): the BPA delegates the transmission and reception of bundles to CSP while preserving BP semantics. Rather than reimplementing the convergence layer in Rust,

Hardy reuses the same C `libcspcl` library that the other BPAs employ. A dedicated `cspcl` crate wraps the library through an FFI binding crate, distributed with CSPCL itself, that pairs a `bindgen`-generated low-level layer with a safe, asynchronous wrapper adapting the synchronous C interface to Hardy's Tokio runtime without blocking executor threads. Hardy exposes a `ClA` Rust trait whose principal methods are `on_register` (which supplies a back-channel for delivering received bundles into the BPA) and `forward` (which sends a bundle outward). The `cspcl` crate implements this trait, and CSP becomes a first-class address type alongside TCP within the BPA's address enumeration. A lightweight runtime drives both directions over the chosen CSP interface (loopback or a CAN bus): outbound, `forward` hands the serialised bundle to the C library for transmission; inbound, a dedicated receive loop delivers each received bundle directly to the BPA.

IV. CHARON: IP/CAN PROXY OVER DTN

Charon is a lightweight IP, CSP and CAN network tunnel designed to abstract the complexities of Delay/Disruption Tolerant Networking (DTN). Its goal is to make legacy communications using CSP delay and disruption-tolerant without requiring direct integration with DTN protocols. Charon supports three protocols to proxy : IP, CAN & CSP. IP and CAN modes support packet encapsulation, whereas in CSP mode Charon acts like a local proxy between the two peers. Charon relies on Protobuf for serialization and data handling on the BPA side thanks to AAP2 [18] and `libcsp` [19] in order to support the CSP stack.

Charon encapsulates IP packets and sends them to the other peer through a BPA. This way, in-satellite applications don't need to integrate the Bundle Protocol directly in order to communicate with a ground station. Thanks to the encapsulation of the network packets in bundles, they still benefit from the features provided by Delay/Disruption Tolerant Networks.

As any other network tunnels, Charon acts like a pipe for network packets. It receives packets from a virtual network interface and forwards them to a Bundle Protocol Agent. For IP, the network interfaces are virtualized thanks to TUN. For example, Charon supports the encapsulation of IP packets using TUN, which provides a Linux device node at `/dev/net/tun`. This allows developers to interact with virtual IP network interfaces. Charon reads from this node, gaining access to a virtual interface, and forwards all packets captured by the interface to the Bundle Protocol Agent by encapsulating them as Application Data Units (ADUs) within bundles. The encapsulated packet then traverses the DTN topology until it reaches the destination (see Figure 1). There, the receiving Charon peer extracts the ADU, decapsulates the packet, and writes it directly to its own TUN interface. As a result, the peer receives the packet as if it had been sent over a standard IP socket.

Finally, Charon supports communication between two CSP peers. However, `libCSP` v1.6, the implementation of the Cube-Sat Space Protocol used by the CSUM, does not provide

virtual network interfaces like TUN or VCAN. This design limitation makes traffic encapsulation challenging, as CSP packets cannot be reliably intercepted outside the process running the CSP socket itself. To address this, we implemented a workaround: Charon acts as a CSP peer-to-peer association server, enabling communication between CSP peers through a DTN topology.

Consider a peer, Alice, that wants to communicate with a second peer, Bob. Alice initiates a connection with its local instance of Charon. Alice's Charon forwards all incoming packets from Alice to its BPA through AAP2. Bob's BPA receives the bundle containing Alice's message, forwards it to Bob's Charon instance, which then transmits the message to Bob.

This architecture enables peer-to-peer communication over CSP in a DTN topology. However, it does not support the routing features provided by CSP, as each peer only interacts with its local Charon instance.

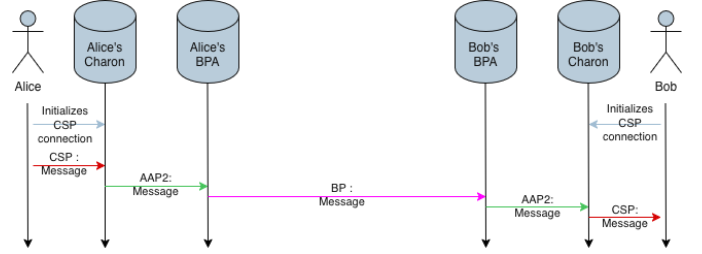


Fig. 1. Flow diagram of the CSP mode in Charon

The current version of Charon only supports μ D3TN as the BPA, μ D3TN provides a well documented and easy to integrate protocol to communicate with the BPA : AAP2 [18]. AAP2 is a Protocol Buffers (protobuf)-based API for interacting with Bundle Protocol Agents (BPAs). It serves as the primary interface for sending bundles via μ D3TN. As a result, Charon is directly compatible with any BPA that supports AAP2 and transitively compatible with all BPAs that μ D3TN can interoperate with.

In the future, we are planning to add support for other Bundle Protocol Agents (BPAs), such as Unibo-BP, ION-DTN or Hardy. Currently, Charon lacks a local packet queue, as it relies entirely on the front-end BPA to handle outgoing packets. This dependency creates a single point of failure: if the BPA goes down, all packets are lost. To address this, integrating a local buffer would mitigate the risk and ensure packets are retained until the BPA recovers.

Charon supports CAN tunneling as well. The CAN mode leverages SocketCAN's [20] VCAN interface to achieve a similar abstraction for CAN frames. Charon opens a socket on a VCAN interface using SocketCAN along with the `CAN_RAW` flag. This way, Charon can receive and write raw CAN frames. As described previously, Charon forwards packets to the BPA and listens for incoming ADUs in order to write their data to the CAN socket. The CAN encapsulation is currently implemented in Charon. However, there is a specific case that could

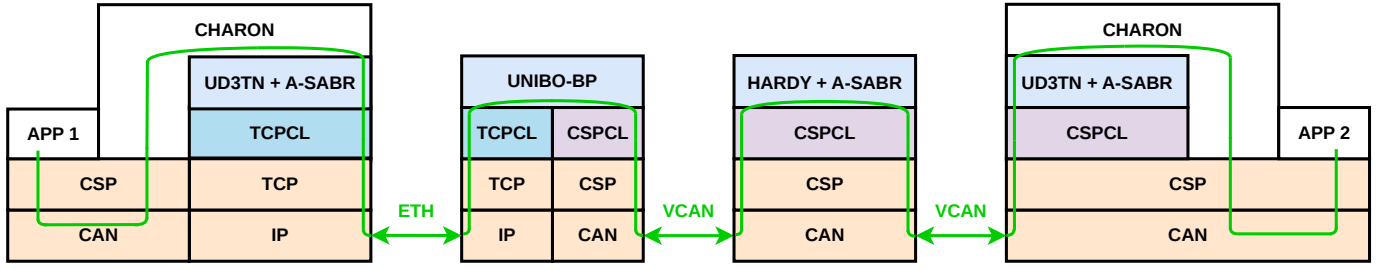


Fig. 2. Experimental setup

make communication between two peers using Charon in CAN mode not disruption or delay tolerant. A standard CAN frame carries about 8 bytes of data, whereas a CSP packet can be much larger. To accommodate this, CSP fragments the packet into multiple CAN frames, which are then sent over the bus. However, in a DTN context, if the CAN frames of a single CSP packet are disseminated over 2 bundles, the BPA might route the bundles differently, potentially breaking the reassembly of the packet on the receiving peer due to frame ordering issues. The solution to this problem is to reassemble CSP packets from the CAN frames intercepted by Charon, ensuring that the delivery order is preserved even in a disrupted network.

V. EXPERIMENTAL SETUP

The integration was validated on flight-representative hardware: two PolarFire RISC-V boards of Microchip linked by a CAN bus, on which two communicating Hardy nodes were successfully deployed. The setup is described further is slightly different and uses VCAN interfaces for convenience.

A. Network Topology

The end-to-end validation topology forms a linear four-node chain in which each hop uses a distinct transport and BPA combination, exercising the full heterogeneity of the stack:

- 1) **Node 0 (alice)** μ D3TN, EID named `dtn://alice.dtn/`, acting as the ingress gateway. It exposes an AAP2 socket to Charon and connects outward to Node 1 over TCPCLv3. An A-SABR Bundle Dispatch Manager (BDM) attaches to μ D3TN over AAP2 and makes all forwarding decisions for traffic entering the network at this node.
- 2) **Node 1 (unibo)** Unibo-BP, EIDs named `dtn://unibo.dtn/` and `ipn:1.0`. It receives bundles from Node 0 via TCPCLv3 and forwards them over the virtual CAN bus using the `unibo-bp-cspcl` daemon (CSP address 1).
- 3) **Node 2 (hardy)** Hardy, EIDs named `dtn://hardy.dtn/` and `ipn:2.0`. It communicates with both neighbouring nodes over the virtual CAN bus via its built-in CSPCL CLA (CSP address 2). A-SABR is integrated directly into Hardy as its routing agent.
- 4) **Node 3 (bob)** μ D3TN, EID named `dtn://bob.dtn/`, acting as the egress gateway. It receives bundles from

Node 2 over the virtual CAN bus via the CSPCL CLA (CSP address 3) and delivers them to Charon over AAP2. An A-SABR BDM drives its forwarding table.

The full four-node experiment reported here runs entirely in software, with the CAN interfaces replaced by virtual CAN (`vcn0`) and the TCP segment exercised on the development host.

At both ends of the chain, a Charon instance bridges IP applications to the DTN layer. Charon alice encapsulates IP datagrams received on a TUN interface into BP bundles addressed to `dtn://bob.dtn/charon` and injects them into Node 0 via AAP2. Charon bob performs the inverse operation, decapsulating received bundles and injecting the recovered IP datagrams into a second TUN interface. This arrangement allows unmodified IP applications to communicate transparently through the DTN overlay without any awareness of the underlying store-and-forward path.

B. Routing and Contact Plan

A-SABR runs as the routing algorithm on Node 0 and 3 (alice and bob) and Node 2 (hardy). Those instances share a common contact plan defining four nodes and six bidirectional contact entries. The alice-unibo hop is modelled with a nominal rate of 100 kbps and negligible propagation delay, reflecting the TCP link. The two CAN hops (unibo-hardy and hardy-bob) are each modelled at 50 kbps with a 5 ms one-way light-time, consistent with the CAN bus bandwidth budget. All contacts span the full experiment window with no scheduled disruptions, making this a connectivity validation rather than a disruption-tolerance test.

C. Hardware Platform Validation

The physical validation platform consists of two PolarFire SoC RISC-V development boards connected by a CAN bus. This configuration mirrors the avionics architecture of the Centre Spatial de l'Université de Montpellier's upcoming 12U CubeSat mission, which uses CAN as the primary inter-board communication medium. The CAN interface is exposed to the Linux operating system through the SocketCAN subsystem, allowing standard `ip link` tooling to manage the bus and the CSP stack to bind to it directly using the `can_socketcan` driver. The PolarFire were used to test and validate the communication between two Hardy nodes using A-SABR for routing the bundles. They also validated that two Unibo nodes were communicating with each other.

VI. FUTURE WORK

The target 12U platform is linux-based, although other CubeSats from the CSUM are micro-controller-based. A-SABR is currently being ported for micro-controller support, but not the CSPCL stack. Future work includes a full experiment on microcontroller, with a compatible Bundle Protocol Agent. Moreover, tunneling CAN frames over the bundle protocol is not satisfactory and the tunneling of CSP packets is being developed at the time writing.

REFERENCES

- [1] V. Cerf, S. Burleigh, A. Hooke, L. Torgerson, R. Durst, K. Scott, K. Fall, and H. Weiss, "Delay-tolerant networking architecture," Internet Requests for Comments, RFC Editor, RFC 4838, April 2007. [Online]. Available: <http://www.rfc-editor.org/rfc/rfc4838.txt>
- [2] "The future mars communications architecture," *Interagency Operations Advisory Group*, 2022. [Online]. Available: <https://www.ioag.org/Public%20Documents/MBC%20architecture%20report%20final%20version%20PDF.pdf>
- [3] "LCRNS Project - TEMPO," <https://tempo.gsfc.nasa.gov/projects/LCRNS/>, 2023, accessed: 2024-03-15.
- [4] P. Giordano, A. Grenier, P. Zoccarato, L. Bucci, A. Cropp, R. Swinden, D. Gomez Otero, W. El-Dali, W. Carey, L. Duvet *et al.*, "Moonlight navigation service-how to land on peaks of eternal light," in *Proceedings of the 72nd International Astronautical Congress (IAC), Dubai, United Arab Emirates*, 2021, pp. 25–29.
- [5] "ESA's Moonlight: Connectivity and Secure Communications," https://www.esa.int/Applications/Connectivity_and_Secure_Communications/Moonlight, 2023, accessed: 2024-03-15.
- [6] D. J. Israel, K. D. Mauldin, C. J. Roberts, J. W. Mitchell, A. A. Pulkkinen, D. C. La Vida, M. A. Johnson, S. D. Christe, and C. J. Gramling, "Lunanet: a flexible and extensible lunar exploration communications and navigation infrastructure," in *2020 IEEE Aerospace Conference*. IEEE, 2020, pp. 1–14.
- [7] P. Giordano, R. Swinden, C. Gramling, J. Crenshaw, and J. Ventura-Traveset, "Lunanet position, navigation, and timing services and signals, enabling the future of lunar exploration," in *Proceedings of the 36th International Technical Meeting of the Satellite Division of The Institute of Navigation (ION GNSS+ 2023)*, 2023, pp. 3577–3588.
- [8] NASA, "NASA's LunaNet: Empowering Artemis with Communications and Navigation Interoperability," 2023, [Online]. Available: <https://www.nasa.gov/humans-in-space/lunanet-empowering-artemis-with-communications-and-navigation-interoperability/>, Accessed: Mar. 15, 2024.
- [9] NASA, "Lunanet interoperability specification version 5 - draft," <https://www.nasa.gov/wp-content/uploads/2023/09/lunanet-interoperability-specification-v5-draft.pdf?emrc=6f4483>, 2023, accessed: Mar. 15, 2024.
- [10] K. Scott, S. Burleigh, and E. Birrane, "Bundle protocol version 7," Internet Requests for Comments, RFC Editor, RFC 9171, January 2022, <http://www.rfc-editor.org/rfc/rfc9171.txt>. [Online]. Available: <http://www.rfc-editor.org/rfc/rfc9171.txt>
- [11] Consultative Committee for Space Data Systems (CCSDS), "Ccsds bundle protocol specification (blue book, recommended standard CCSDS 734.2-B-1," <https://public.ccsds.org/Pubs/734x2b1.pdf>, September 2015.
- [12] M. Feldmann, J. A. Fraire, F. Walter, and S. C. Burleigh, "Ring road networks: Access for anyone," *IEEE Communications Magazine*, vol. 60, no. 4, pp. 38–44, 2022.
- [13] D. Evans and M. Merri, "Ops-sat: A esa nanosatellite for accelerating innovation in satellite control," in *SpaceOps 2014 Conference*, 2014, p. 1702.
- [14] O. De Jonckère, L. Ma, and J. A. Fraire, "A-sabr: The adaptive library for schedule-aware bundle routing," in *2025 IEEE International Conference on Wireless for Space and Extreme Environments (WiSEE)*. IEEE, 2025, pp. 1–6.
- [15] B. Book, "Schedule-aware bundle routing," *Consultative Committee for Space Data Systems*, 2019.
- [16] S. Pierrot, O. De Jonckère, S. Burleigh, and C. Geoffroy, "Design and implementation of a native socket interface for the bundle protocol," in *2025 IEEE International Conference on Wireless for Space and Extreme Environments (WiSEE)*, 2025, pp. 1–6.
- [17] C. Caini and L. Persampieri, "Design and features of Unibo-BP, the Unibo implementation of the DTN Bundle Protocol," *IEEE Journal of Radio Frequency Identification*, 2024, early access.
- [18] F. Walter, M. Feldmann, J. A. Fraire, and S. Burleigh, "The architectural refinement of ud3tn: Toward a software-defined dtn protocol stack," in *2024 IEEE 10th International Conference on Space Mission Challenges for Information Technology (SMC-IT)*, 2024, pp. 161–170.
- [19] <https://github.com/libcsp/libcsp>.
- [20] <https://docs.kernel.org/networking/can.html>.